

Conestoga College

School of Applied Computer Science & Information Technology

SENG8081 - Case Studies

Canadian Job Market Analysis

Anuroopa Balachandran

Bin Hu

Ce Chen

Xiaoman Yang

July 31, 2025

Abstract

In today's dynamic job market, access to reliable employment data is critical for governments, employers, and job seekers alike. This report presents our comprehensive system for collecting, validating, and analyzing Canadian labor market information including labour force statistics and industry level employment data from official repositories and job scraping data.

The core processing of the system is managed through a modular Python pipeline which processes data from diverse sources such as structured government repositories and scraped job listings. All data undergoes rigorous quality checks including completeness verification and Canadian location validation ensuring clean, consistent outputs. The system features multi-source collection capabilities, automated quality reporting, and storage in a MongoDB database (*job_market_trends*) with specialized collections for labor force metrics, industry statistics, and job postings. Designed for efficiency, it enables seamless queries and dashboard integration for visualization. By combining robust data processing with flexible NoSQL storage, this project offers a scalable, reliable foundation for Canadian labor market research. The following are the main points of emphasis for this Python-based project:

Data Collection:

Data is retrieved from multiple sources including Statistics Canada CSVs and scraped job postings. These sources provide a mix of historical and up-to-date job market information.

Data Quality:

The data quality management layer of the system includes a separate validation module that runs basic checks on data before it enters the final storage layer.

Data Storage:

The storage layer loads validated datasets into a MongoDB instance. Within MongoDB, there are several topic-based collections, including scraped job listings and job statistics. The system is designed to support flexible queries and integration with visualization for analysis.

Data Visualization:

Visualizations turn labour data into easy-to-understand charts that reveal hiring trends, regional variations, and shifting occupations. Powered by live data from MongoDB and built with Python, these tools let you explore employment changes as they happen.

Contents

1.	Introduction	4
1.1	Data Retrieval	4
1.2	Data Management	4
2.	System Diagram.....	5
3.	Data Research and Integration.....	6
3.1	Problem statement	6
3.2	Statistics Canada – Labour Force Characteristics.....	6
3.3	Statistics Canada – Employment by Industry	7
3.4	Open Canada – Job Opening Projections (2025–2033).....	7
3.6	Job Bank Canada – Live Job Postings.....	8
3.7	Data Management and Integration.....	8
4.	Data Collection.....	9
4.1	Data Collection Process:.....	9
4.2	Challenges Faced for collection.....	11
4.3	Data Preprocessing/Cleaning	11
5.	Data Storage and Maintenance	12
5.1	Storage	13
5.2	Why MongoDB?.....	13
5.3	Maintenance	14
6.	Data Quality	15
6.1	Accuracy.....	15
6.2	Completeness.....	16
6.3	Consistency	16
6.4	Reliability	16
7.	Data Analysis and Visualization.....	18
7.1	Data Visualizations	18
8.	DevOps	28
9.	Extension	29
10.	Allocation Project Team Roles.....	30
11.	Project Timeline.....	30
	References	31

1. Introduction

This project cleanses and combines various sources of labor data into a cohesive data ecosystem to gain insights about the Canadian job market, from public data sources such as historical labor force data and industry employment data to real-time job listings. The aim is to provide insight on employment and job growth by gender, industry sectors, and regions in Canada, while also ensuring data quality, consistency, and scalability. The resulting system is used by decision-makers, analysts, and job seekers to gain accurate and relevant information about the Canadian labor market.

1.1 Data Retrieval

Public Data Sources: Historical labor force data, industry employment data, and long-term labor projections are downloaded from public sources such as Statistics Canada and Government of Canada websites. These data sources form the basis for trend data by gender, age, industry sectors, and regions.

Real-Time Job Data: In addition to historical records, we also download data on current job listings from public job listing websites and job listing APIs to ensure that the pipeline reflects new job roles that have emerged, new salary ranges for existing jobs, and current labor demands.

Automated Collection Scripts: Python scripts automate the download, preprocessing, and organization of datasets. These scripts ensure consistent file naming, directory structure, and regular refresh schedules, enhancing the reproducibility of the data pipeline.

1.2 Data Management

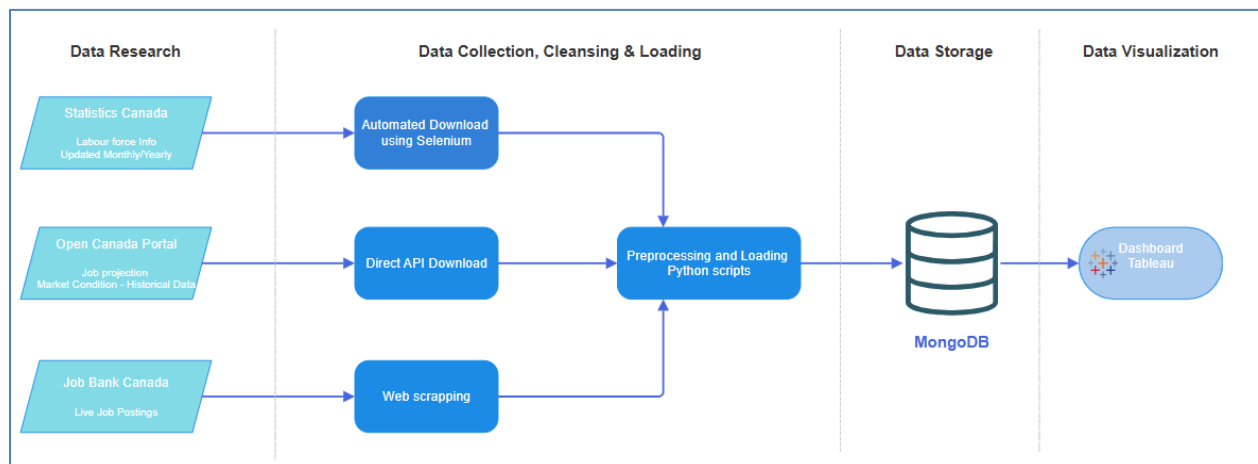
Flexible Storage Using MongoDB: All collected datasets are stored in a document-based database, MongoDB, which allows schema flexibility and simplifies integration of diverse data types, including unstructured job descriptions and structured labor force tables.

Data Validation & Quality Assurance: A dedicated module performs quality checks on real-time data, verifying completeness of critical fields (e.g., job title, company, location), and ensuring geographic accuracy. Invalid entries are flagged, summarized, and reported to maintain data reliability.

Integration and Access: The MongoDB structure supports seamless integration with downstream applications such as visualization dashboards (Tableau) and analytical tools. With efficient indexing and aggregation pipelines, users can easily query by sector, region, or posting date.

In summary, the Canadian Job Market Analysis project provides a robust framework for collecting, storing, validating, and analyzing labor data. By combining historical depth with real-time intelligence, the system equips users with a dynamic and trustworthy view of employment patterns across Canada.

2. System Diagram



The following diagram illustrates the multi-stage process developed to aggregate, preprocess, store, and visualize Canadian labour market data from diverse sources.

Component 1: Data Research

We began by identifying and cataloging key sources of labour market information:

- Statistics Canada: Monthly and annual tables covering labour force characteristics and employment by industry.
- Open Canada Portal: API-accessible datasets on job opening forecasts and market conditions.
- Job Bank Canada: Dynamic listings of live job postings with occupational codes and employer metadata.

Component 2: Data Collection, Cleansing & Loading

- Automated scripts were built for each source using appropriate tools:
- Selenium for browser-based downloads from Statistics Canada
- Requests Library for API calls to Open Canada
- BeautifulSoup + Selenium for web scraping Job Bank Canada
- All raw inputs were funneled through preprocessing modules that standardized formats, validated values, and loaded clean datasets into storage.

Component 3: Data Storage

Cleaned data was organized into five MongoDB collections such as *labour_force_stats*, *industry_employment*, *employment_forecast*, *market_condition*, and *job_postings*. Indexes were applied on key fields such as *ref_date*, *geo*, and *noc_code* to support efficient queries and dashboard filtering.

Component 4: Visualization Layer

Tableau dashboards were connected via the MongoDB BI Connector to enable interactive filtering by geography, occupation, and timeframe. These visual tools allow users to explore workforce trends, regional disparities, and hiring dynamics in near real-time.

3. Data Research and Integration

3.1 Problem statement

Canada's labor market is undergoing continuous transformation influenced by economic shifts, emerging industries, regional demands, and evolving workforce skills. However, navigating these trends remains complex due to fragmented data sources, inconsistent formats, and rapidly changing job landscapes. This project aims to centralize, clean, and analyze this data to reveal key trends across industries, occupations, regions, and time.

We have collected quite a lot of datasets from multiple sources to understand labor market conditions across time, industries, and geographies. After careful observation, we finalized the following key sources to support our project objectives:

3.2 Statistics Canada – Labour Force Characteristics

Source: <https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410028703>

This dataset provides monthly labour force statistics across Canadian provinces, segmented by

gender and age groups. It covers employment levels, unemployment rates, and participation metrics, offering crucial insight into population-level workforce trends from 2021 to 2025.

Process:

- Use Selenium automation to programmatically access monthly data across multiple years.
- Download and merge CSV files representing different months into a unified dataset using Pandas.
- Save the combined dataset as StatCan_Labourforce.csv for downstream analysis.

3.3 Statistics Canada – Employment by Industry

Source: <https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002301>

This dataset includes industry-wise monthly employment statistics across provinces, following the NAICS classification. It supports analysis of employment dynamics across sectors like manufacturing, healthcare, education, and services.

Process:

- Loop through 11 provinces using Selenium to download industry-specific employment files.
- Merge all regional datasets into a single CSV using Pandas, saved as StatCan_byIndustry.csv.
- Structure the data to support comparisons across sectors and provinces over time.

3.4 Open Canada – Job Opening Projections (2025–2033)

Source: <https://open.canada.ca/data/en/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/da1135c5-a1df-4a07-a81e-de308ae6cce6>

This forward-looking dataset estimates projected job openings by occupation across Canada, driven by factors such as retirements, growth, and labour mobility. It enables long-term workforce planning.

Process:

- Use the requests library to download the official CSV file directly via API.
- Save the file as employment_projections_2024_2033.csv.
- Integrate the projections into your trend analysis and forecasting models.

3.5 Open Canada – Labour Market Conditions (2021–2023)

Source: <https://open.canada.ca/data/en/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/ff795791-0728-4dcf-8fdd-d7ba9e1210b8>

This dataset assesses recent labour market conditions for various occupations, categorized as “shortage,” “balance,” or “surplus.” It helps determine the demand-supply balance of specific skills across regions.

Process:

- Fetch the CSV file via a GET request using Python's requests module.
- Save and rename the file as labour_market_conditions_2021_2023.csv.
- Merge it with occupation and regional job demand data for insight into pressure points in the job market.

3.6 Job Bank Canada – Live Job Postings

Source: <https://www.jobbank.gc.ca/jobsearch/>

A continuously updated repository of active job openings across Canada, offering real-time insights into hiring trends by **industry**, **region**, and **occupation**. It supports career planning, labour market analysis, and job matching for individuals and employers.

Process:

- Use the Job Bank web scraping tools to extract live job postings data.
- Filter by location, occupation, industry, or employment type to tailor insights.
- Analyze posting frequency, job titles, and regional demand to identify high-growth sectors.
- Integrate with other for deeper workforce analysis.
- Visualize trends using Tableau or Power BI to support career guidance or policy decisions.

3.7 Data Management and Integration

After acquiring all five datasets, the project integrates them as follows:

Database Schema

A modular MongoDB schema has been designed to manage labour market data, including time-series statistics, industry-sector details, occupation-based conditions, and future projections. Each dataset is stored in a separate collection, with references and indexing strategies enabling unified, scalable analysis across domains like region, occupation, and time.

Synchronization & Preprocessing

The Selenium-based downloads for monthly StatCan data are rerun periodically to reflect recent months. Open Canada datasets are less frequently updated and refreshed manually or via scheduled scripts. The job bank data sets are updated by scrapping from the website hourly basis. Merged datasets are cleaned, normalized, and transformed using Pandas.

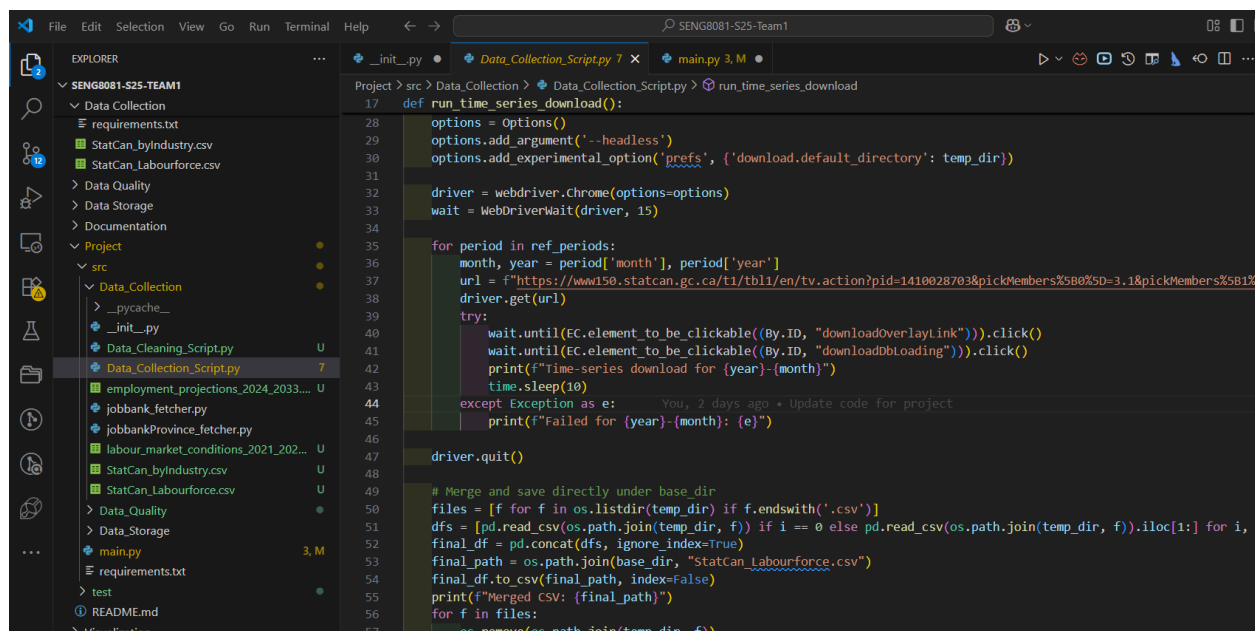
Integration Pipeline: Final outputs are stored in unified CSV files in the Data Collection directory. These are read into dataframes for further use in EDA, visualizations, and dashboard tools.

4. Data Collection

4.1 Data Collection Process:

Step 1: Collection of Labour Force Characteristics (StatCan Table 14-10-0287-03)

- We used Python with Selenium to automate downloading monthly labour force data from Statistics Canada's Table. The script loops through specific months and years (from 2021 to 2025) and triggers CSV downloads from the StatCan portal.
- All downloaded CSVs are loaded using *pandas.read_csv()* and merged into a single DataFrame for preprocessing and cleaning.
- Temporary files are removed after merging, and the consolidated file is saved as StatCan_Labourforce.csv.



```
File Edit Selection View Go Run Terminal Help
SENG8081-S25-Team1
EXPLORER
SENG8081-S25-TEAM1
  Data Collection
    requirements.txt
    StatCan_byIndustry.csv
    StatCan_Labourforce.csv
  Data Quality
  Data Storage
  Documentation
  Project
    src
      Data_Collection
        __pycache__
        _init_.py
        Data_Cleaning_Script.py
        Data_Collection_Script.py
        employment_projections_2024_2033...
        jobbank_fetcher.py
        jobbankProvince_fetcher.py
        labour_market_conditions_2021_202...
        StatCan_byIndustry.csv
        StatCan_Labourforce.csv
      Data_Quality
      Data_Storage
      main.py
      requirements.txt
      test
      README.md
      Visualization
  Data_Collection_Script.py 7
  main.py 3. M
  run_time_series_download

Project > src > Data_Collection > Data_Collection_Script.py > run_time_series_download
17 def run_time_series_download():
18     options = Options()
19     options.add_argument('--headless')
20     options.add_experimental_option('prefs', {'download.default_directory': temp_dir})
21
22     driver = webdriver.Chrome(options=options)
23     wait = WebDriverWait(driver, 15)
24
25     for period in ref_periods:
26         month, year = period['month'], period['year']
27         url = f"https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410028703&pickMembers%5B0%5D=3.1&pickMembers%5B1%5D=2021-12-01"
28         driver.get(url)
29         try:
30             wait.until(EC.element_to_be_clickable((By.ID, "downloadOverlayLink"))).click()
31             wait.until(EC.element_to_be_clickable((By.ID, "downloadDbLoading"))).click()
32             print(f"Time-series download for {year}-{month}")
33             time.sleep(10)
34         except Exception as e:
35             print(f"Failed for {year}-{month}: {e}")
36
37     driver.quit()
38
39     # Merge and save directly under base_dir
40     files = [f for f in os.listdir(temp_dir) if f.endswith('.csv')]
41     dfs = [pd.read_csv(os.path.join(temp_dir, f)) if i == 0 else pd.read_csv(os.path.join(temp_dir, f)).iloc[1:] for i, f in enumerate(files)]
42     final_df = pd.concat(dfs, ignore_index=True)
43     final_path = os.path.join(base_dir, "StatCan_Labourforce.csv")
44     final_df.to_csv(final_path, index=False)
45     print(f"Merged CSV: {final_path}")
46     for f in files:
47         os.remove(os.path.join(temp_dir, f))
```

Step 2: Collection of Employment by Industry (StatCan Table 14-10-0023-01)

- Similar automation via Selenium was implemented to download data across 11 provinces and territories. Each regional CSV is downloaded and merged using pandas.
- Output is saved as StatCan_byIndustry.csv for industry-level employment trend analysis.

```

60 # Labour force characteristics by industry
61 def run_geography_download():
62     geographies = [{"id": f"1.{i}", "name": name} for i, name in enumerate([
63         "Canada", "Newfoundland and Labrador", "Prince Edward Island", "Nova Scotia",
64         "New Brunswick", "Quebec", "Ontario", "Manitoba", "Saskatchewan",
65         "Alberta", "British Columbia"
66     ], start=1)]
67
68     temp_dir = os.path.join(base_dir, "temp_geo")
69     os.makedirs(temp_dir, exist_ok=True)
70
71     options = Options()
72     options.add_argument('--headless')
73     options.add_experimental_option('prefs', {'download.default_directory': temp_dir})
74
75     driver = webdriver.Chrome(options=options)
76     wait = WebDriverWait(driver, 15)
77
78     for geo in geographies:
79         geo_num = geo['id'].split('.')[1]
80         url = f"https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002301&pickMembers%5B0%5D=1.{geo_num}"
81         driver.get(url)
82         try:
83             wait.until(EC.element_to_be_clickable(By.ID, "downloadOverlayLink")).click()
84             wait.until(EC.element_to_be_clickable(By.ID, "downloadDbLoading")).click()
85             print(f"Geography download for {geo['name']}")
86             time.sleep(10)
87         except Exception as e:
88             print(f"Failed for {geo['name']}: {e}")
89
90     driver.quit()
  
```

Step 3: Direct API Downloads (Open Canada Portal)

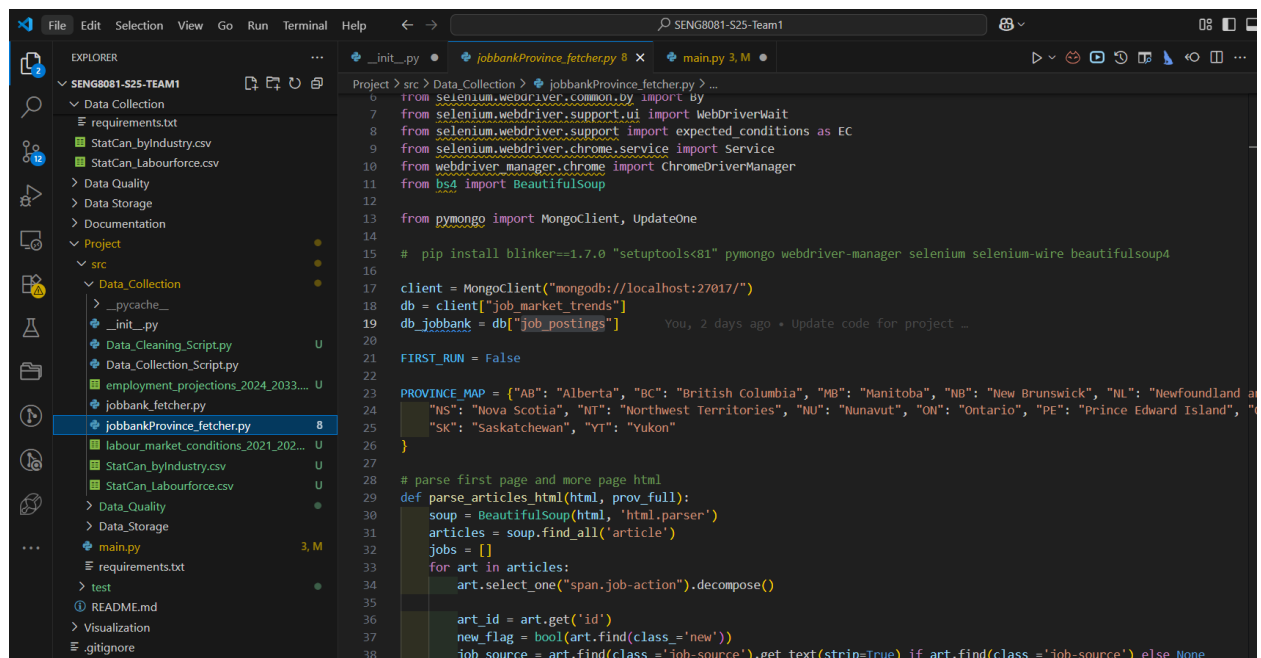
- A CSV file is directly retrieved using the requests library from Open Canada's API. The dataset estimates future job openings by NOC code and region, considering factors like retirements and employment growth.
- Another dataset is downloaded from Open Canada's API, capturing labour market status (surplus, shortage, balance) for each occupation and region over the past three years.
- These CSVs are directly saved, later loaded into the system using pandas for integration.

```

103 # Direct API Downloads
104 def run_api_downloads():
105     resources = [
106         {
107             "name": "employment_projections_2024_2033.csv",
108             "url": "https://open.canada.ca/data/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/da1135c5-a1df-4a07-a
109         },
110         {
111             "name": "labour_market_conditions_2021_2023.csv",
112             "url": "https://open.canada.ca/data/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/ff795791-0728-4dcf-0
113         }
114     ]
115
116     for res in resources:
117         path = os.path.join(base_dir, res["name"])
118         try:
119             resp = requests.get(res["url"])
120             resp.raise_for_status()
121             with open(path, "wb") as f:
122                 f.write(resp.content)
123             print(f"API download complete: {res['name']}")
124         except Exception as e:
125             print(f"Failed to download {res['name']}: {e}")
126
  
```

Step 4: Live Job Postings (Job Bank Canada)

- Web Scrapping: Job Bank Canada provides real-time job posting data. Team prepared a scraping script to extract regional job demand by NOC codes. This script involves Selenium, BeautifulSoup, or a future API endpoint.
- Tools used are Python - Pandas, Requests, Selenium with ChromeDriver (Headless).
- CSV file handling and merging.



```
Project > src > Data_Collection > jobbankProvince_fetcher.py > ...
6 from selenium.webdriver.common.by import By
7 from selenium.webdriver.support.ui import WebDriverWait
8 from selenium.webdriver.support import expected_conditions as EC
9 from selenium.webdriver.chrome.service import Service
10 from webdriver_manager.chrome import ChromeDriverManager
11 from bs4 import BeautifulSoup
12
13 from pymongo import MongoClient, UpdateOne
14
15 # pip install blinker==1.7.0 "setuptools<81" pymongo webdriver-manager selenium selenium-wire beautifulsoup4
16
17 client = MongoClient("mongodb://localhost:27017/")
18 db = client["job_market_trends"]
19 db_jobbank = db["job_postings"]
20
21 FIRST_RUN = False
22
23 PROVINCE_MAP = {"AB": "Alberta", "BC": "British Columbia", "MB": "Manitoba", "NB": "New Brunswick", "NL": "Newfoundland and Labrador", "NS": "Nova Scotia", "NT": "Northwest Territories", "NU": "Nunavut", "ON": "Ontario", "PE": "Prince Edward Island", "QC": "Quebec", "SK": "Saskatchewan", "YT": "Yukon"}
24
25 }
26
27 # parse first page and more page html
28 def parse_articles_html(html, prov_full):
29     soup = BeautifulSoup(html, 'html.parser')
30     articles = soup.find_all('article')
31     jobs = []
32     for art in articles:
33         art.select_one("span.job-action").decompose()
34
35         art_id = art.get('id')
36         new_flag = bool(art.find(class_='new'))
37         job_source = art.find(class_='job-source').get_text(strip=True) if art.find(class_='job-source') else None
```

4.2 Challenges Faced for collection

- StatCan download pages required dynamic interaction, making basic requests unusable. Selenium was required to simulate user interaction for downloading.
- Data formatting inconsistencies (e.g., extra headers, missing values) across years and regions required careful handling during CSV merging.
- Live job posting access from Job Bank is not straightforward, as there's no public API.

4.3 Data Preprocessing/Cleaning

```
def clean_csv(filename, output_path=None, zip_csv_name=None, compress=False,
32
33     # Trim head/tail rows
34     data_lines = lines[skip_head:len(lines) - skip_tail if skip_tail > 0 else None]
35     cleaned_text = "\n".join(data_lines)
36
37     # Load to DataFrame
38     df = pd.read_csv(io.StringIO(cleaned_text))
39     print(f"Original shape: {df.shape}")
40
41     # Standardize column names
42     df.columns = [col.strip().lower().replace(" ", "_") for col in df.columns]
43
44     # Remove rows/columns with ALL missing values
45     df.dropna(axis=0, how='all', inplace=True)
46     df.dropna(axis=1, how='all', inplace=True)
47
48     # Remove columns with all empty strings or whitespace
49     for col in df.columns:
50         if df[col].dtype == object and all(df[col].astype(str).str.strip() == ""):
51             df.drop(columns=[col], inplace=True)
52
53     # Remove duplicate rows
54     df.drop_duplicates(inplace=True)
55
56     # Strip string column whitespace
57     str_cols = df.select_dtypes(include="object").columns
58     df[str_cols] = df[str_cols].apply(lambda x: x.str.strip())
59
60     # Handle missing values
61     if dropna:
62         df.dropna(inplace=True)
```

Cleaning Strategies:

- Standardizes column names to lowercase with underscores and removes leading or trailing spaces.
- Drops rows and columns that contain only missing values, either drops rows with missing data or fills them, using column mean, median, or the placeholder "Unknown", based on configuration.
- Removes columns that contain only blank strings or whitespace characters and deletes any duplicate rows to ensure data uniqueness.
- Trims extra whitespace from all string-type columns.
- It removes non-Unicode columns such as *nom_de_la_profession* from non-StatCan files to clean unwanted attributes.
- It converts column data types to optimized formats using pandas' type inference.

5.Data Storage and Maintenance

For this project, long-term and secure data storage is essential to ensure both regulatory compliance and practical access in the future. We need a solution that supports scalability, flexibility, and efficient data retrieval as our dataset grows.

5.1 Storage

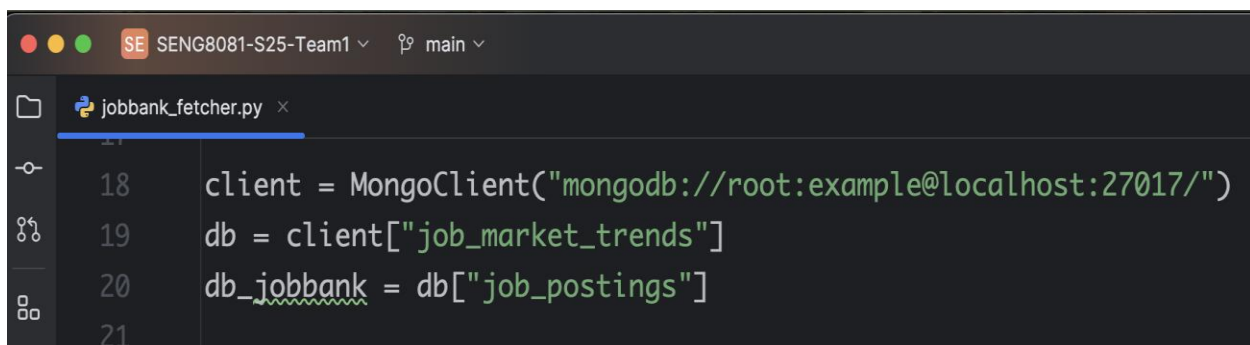
Currently, we are running MongoDB locally in a Docker container for development and testing purposes. This setup provides flexibility and ease of setup during early-stage development. In the future, we plan to migrate to MongoDB Atlas, a fully managed cloud service that offers high availability, automated backups, and built-in tools for data visualization and advanced querying. It also supports features like full-text search and AI-powered capabilities such as vector search, which may be useful as our project evolves.

Note: The project initially used SQL Server due to its structured schema design, transactional reliability, and familiarity for tabular labour data. However, as the dataset scope expanded to include semi-structured sources like scraped job postings and API-based projections, each with inconsistent fields and formats, MongoDB became a more suitable choice. Its schema-free architecture allowed flexible ingestion, rapid iteration, and topic-based collection storage, while also supporting future scalability and direct dashboard integration via the BI Connector, all of which aligned better with evolving project needs.

5.2 Why MongoDB?

We chose MongoDB as our primary database system. A key advantage of MongoDB is its schema-free document model, which is ideal for handling data from diverse and inconsistent sources. Our project integrates data from multiple channels such as public APIs, scraping content, and CSV imports each with its own structure and format. MongoDB allows us to store and query these varied datasets in a flexible way, without enforcing a rigid schema. This greatly simplifies our data ingestion and transformation process, while supporting efficient querying and future scalability.

Python-MongoDB connect:

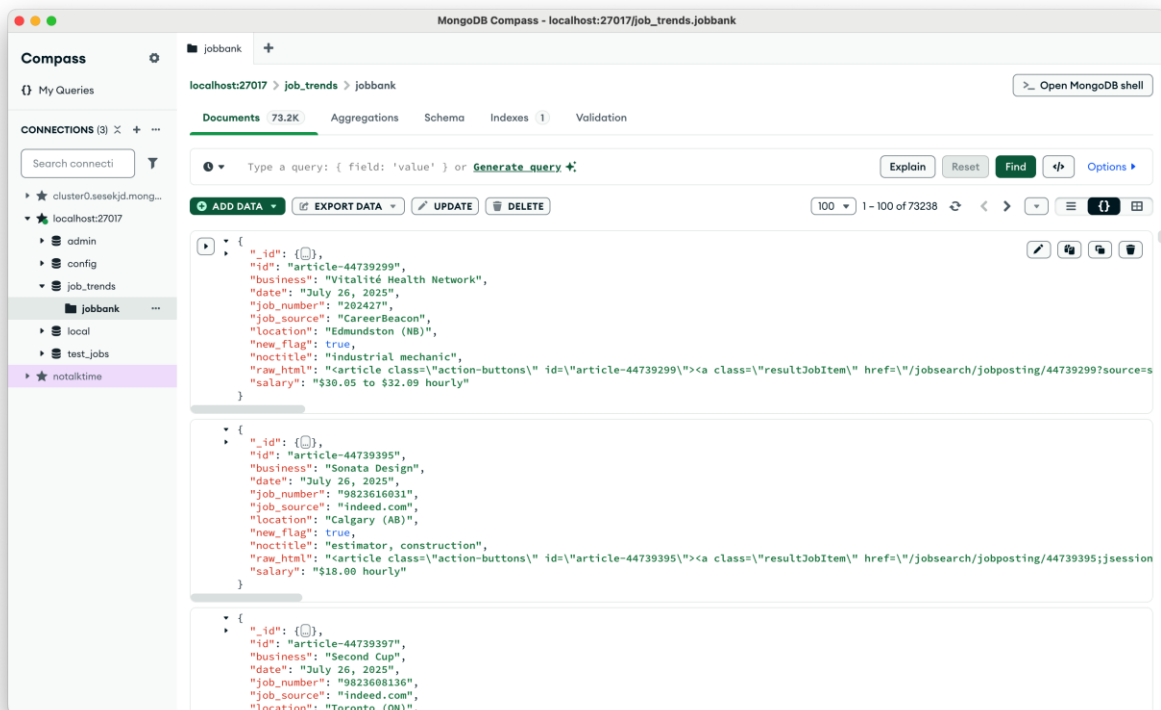
A screenshot of a code editor window. The title bar shows 'SE SENG8081-S25-Team1' and 'main'. The editor has a dark theme. The file name is 'jobbank_fetcher.py'. The code is as follows:

```
18 client = MongoClient("mongodb://root:example@localhost:27017/")
19 db = client["job_market_trends"]
20 db_jobbank = db["job_postings"]
21
```

Python read & write MongoDB:

```
jobbank_fetcher.py
88 def save_jobs(jobs): 2 usages Anuroopa
89     if jobs:
90         ops = [UpdateOne( filter: {"id": job.get("id")}, update: {"$set": job}, upsert=True)
91                     for job in jobs]
92         db_jobbank.bulk_write(ops)
93     count = db_jobbank.count_documents({})
94     print(f"---{len(jobs)} jobs wrote to mongodb, current total: {count}---")
95
96
97 def exist_any_in_db(jobs): 2 usages Anuroopa
98     more_ids = [job.get("id") for job in jobs]
99     exist = db_jobbank.find_one({"id": {"$in": more_ids}})
100     return bool(exist)
101
```

Data in MongoDB:



5.3 Maintenance

Automated database maintenance ensures data integrity, optimizes performance, and scales with your needs, proactively preventing downtime and data loss.

Automated Backups

Scheduled daily and weekly backups will be configured via MongoDB Atlas (post-migration), including point-in-time recovery for critical collections.

Index Management

Indexes on key fields such as *ref_date*, *geo*, and *noc_code* will be monitored and periodically re-evaluated to optimize query performance as data volumes grow.

Data Archiving Policy

Older or infrequently accessed records will be archived into cold storage collections to reduce system load while preserving historical context.

Validation & Integrity Checks

Periodic run of validation scripts to identify duplicate entries, schema drift, or corrupted records, especially in collections sourced from scraping and third-party APIs.

Monitoring & Alerts

Planned integration with Atlas's monitoring dashboard and alert system to flag query slowdowns, storage spikes, and connection failures in real time.

6. Data Quality

Ensuring high-quality data is essential for making reliable decisions in labour market analysis and employment trend forecasting. This report documents the data quality assessment process applied to four cleaned datasets using a Python-based validation framework. The core aspects of data quality such as accuracy, completeness, consistency, and reliability were systematically evaluated to ensure analytical readiness.

6.1 Accuracy

Accuracy refers to the correctness and validity of data values. To assess accuracy in the datasets, the following automated checks were performed:

- Year columns validation: For projections data, columns representing years from 2023 to 2033 were validated to ensure they contain numeric values only. Any deviation may indicate incorrect data types or corrupted entries.
- Numeric fields check: Columns such as *uom_id* and *scalar_id* were required to be numeric across the StatCan datasets. Non-numeric entries were flagged as potential inaccuracies.

These checks help ensure that quantitative analyses based on projections or industry metrics rely on valid numerical values, free from type mismatches or input errors.

6.2 Completeness

Completeness measures whether all required information is present. The script conducted:

- Missing value detection: All datasets were checked for null or missing values across every column. If any row had incomplete data, it was flagged as a failure.
- Presence of required columns: For datasets with temporal dimensions, specific year-based columns were expected. Their absence was treated as a data quality issue.

By ensuring completeness, the analysis avoids skewed insights caused by partial records or dropped values.

6.3 Consistency

Consistency involves having uniform data formats, definitions, and no conflicting entries. The following consistency checks were implemented:

- Duplicate record detection: Any repeated rows in the datasets were identified and flagged.
- Date formatting validation: In the Labourforce dataset, all ref_date values were checked to conform to the YYYY-MM-DD format.
- Consistent geographic values: The geo column was validated against a defined list of valid Canadian provinces and territories. Any mismatches were flagged for correction.

Consistent data enables reliable joins, groupings, and comparisons across datasets, especially when integrating multiple sources.

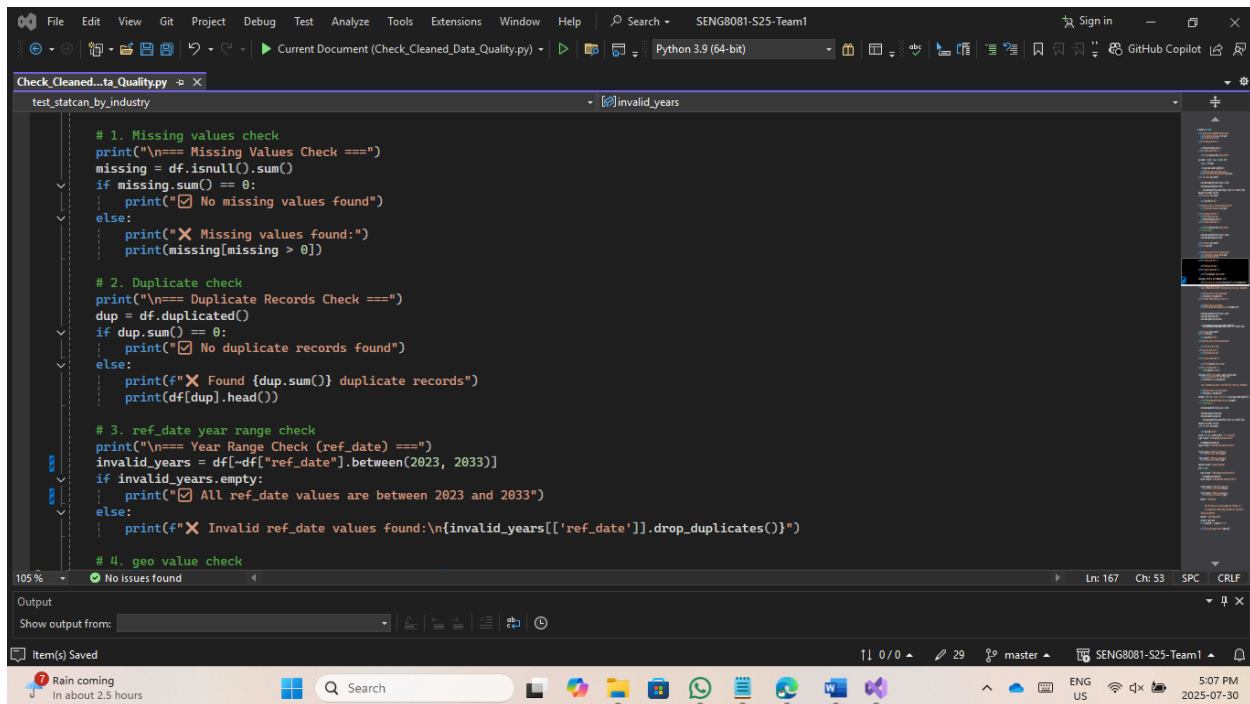
6.4 Reliability

Reliability ensures the trustworthiness of data over time and across systems. The script contributes to reliability by:

- Standardized validation across datasets: Every dataset underwent the same rigorous quality checks tailored to its structure.
- Test result reporting: For each file, a summary was generated indicating the total number of records checked, how many passed all tests, and how many failed, providing clear insight into dataset health.
- Output to HTML reports: Results were written to an HTML-based report, preserving formatting and offering a user-friendly interface to review data quality issues.

This uniform validation and reporting framework increases transparency, and fosters trust in downstream analytical outputs.

Checking Missing Values/Duplicate Records/Year Logic Range:



```
# 1. Missing values check
print("\n=== Missing Values Check ===")
missing = df.isnull().sum()
if missing.sum() == 0:
    print("☑ No missing values found")
else:
    print("✗ Missing values found:")
    print(missing[missing > 0])

# 2. Duplicate check
print("\n=== Duplicate Records Check ===")
dup = df.duplicated()
if dup.sum() == 0:
    print("☑ No duplicate records found")
else:
    print(f"✗ Found {dup.sum()} duplicate records")
    print(df[dup].head())

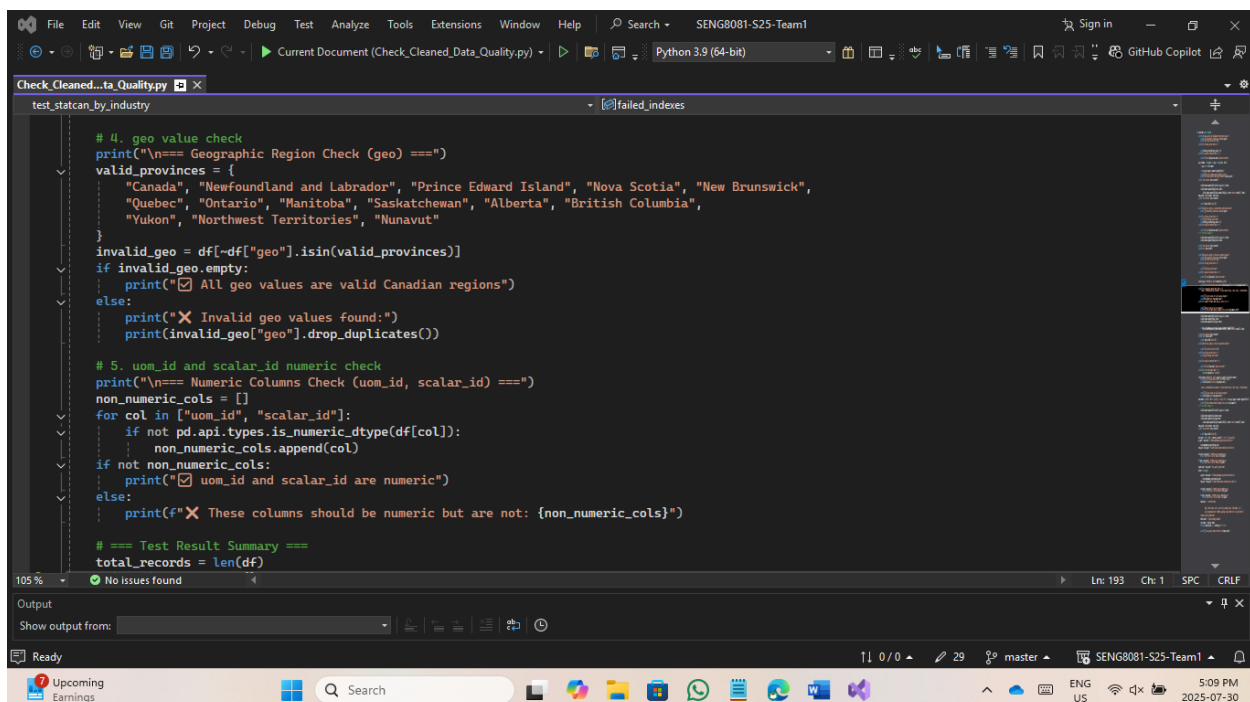
# 3. ref_date year range check
print("\n=== Year Range Check (ref_date) ===")
invalid_years = df[~df["ref_date"].between(2023, 2033)]
if invalid_years.empty:
    print("☑ All ref_date values are between 2023 and 2033")
else:
    print(f"✗ Invalid ref_date values found:\n{invalid_years[['ref_date']].drop_duplicates()}")

# 4. geo value check
print("\n=== Geographic Region Check (geo) ===")
valid_provinces = {
    "Canada", "Newfoundland and Labrador", "Prince Edward Island", "Nova Scotia", "New Brunswick",
    "Quebec", "Ontario", "Manitoba", "Saskatchewan", "Alberta", "British Columbia",
    "Yukon", "Northwest Territories", "Nunavut"
}
invalid_geo = df[~df["geo"].isin(valid_provinces)]
if invalid_geo.empty:
    print("☑ All geo values are valid Canadian regions")
else:
    print("✗ Invalid geo values found:")
    print(invalid_geo["geo"].drop_duplicates())

# 5. uom_id and scalar_id numeric check
print("\n=== Numeric Columns Check (uom_id, scalar_id) ===")
non_numeric_cols = []
for col in ["uom_id", "scalar_id"]:
    if not pd.api.types.is_numeric_dtype(df[col]):
        non_numeric_cols.append(col)
if not non_numeric_cols:
    print("☑ uom_id and scalar_id are numeric")
else:
    print(f"✗ These columns should be numeric but are not: {non_numeric_cols}")

# == Test Result Summary ==
total_records = len(df)
```

Checking Geography/IDs:

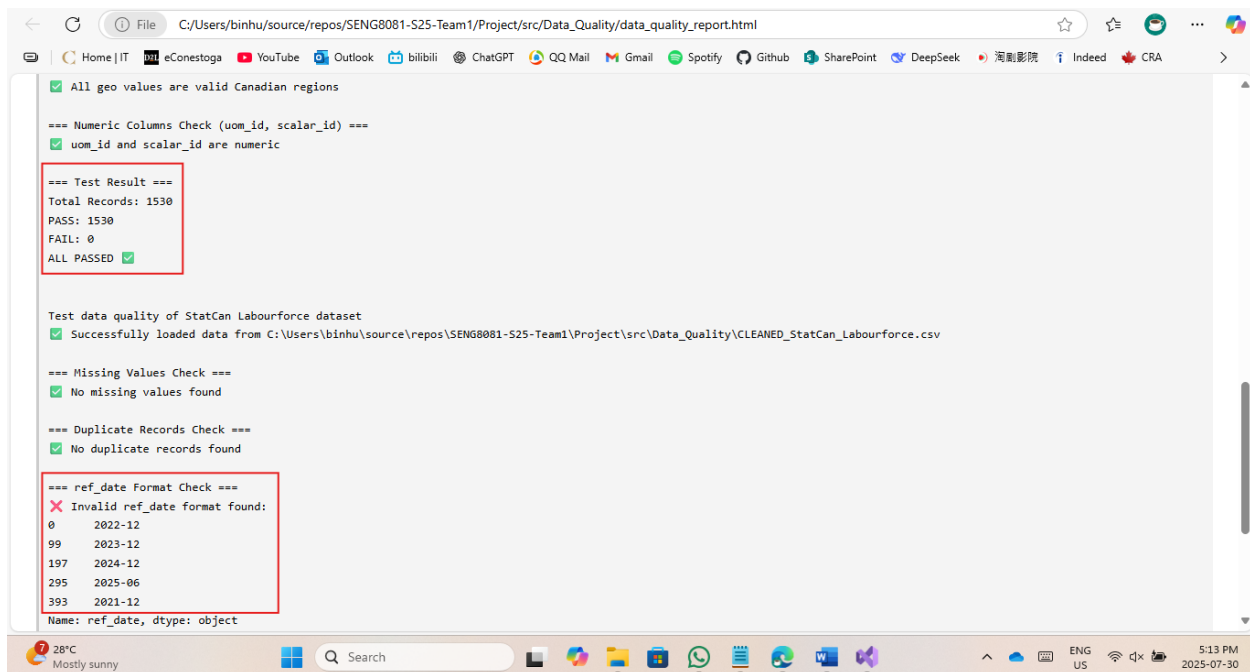


```
# 4. geo value check
print("\n=== Geographic Region Check (geo) ===")
valid_provinces = {
    "Canada", "Newfoundland and Labrador", "Prince Edward Island", "Nova Scotia", "New Brunswick",
    "Quebec", "Ontario", "Manitoba", "Saskatchewan", "Alberta", "British Columbia",
    "Yukon", "Northwest Territories", "Nunavut"
}
invalid_geo = df[~df["geo"].isin(valid_provinces)]
if invalid_geo.empty:
    print("☑ All geo values are valid Canadian regions")
else:
    print("✗ Invalid geo values found:")
    print(invalid_geo["geo"].drop_duplicates())

# 5. uom_id and scalar_id numeric check
print("\n=== Numeric Columns Check (uom_id, scalar_id) ===")
non_numeric_cols = []
for col in ["uom_id", "scalar_id"]:
    if not pd.api.types.is_numeric_dtype(df[col]):
        non_numeric_cols.append(col)
if not non_numeric_cols:
    print("☑ uom_id and scalar_id are numeric")
else:
    print(f"✗ These columns should be numeric but are not: {non_numeric_cols}")

# == Test Result Summary ==
total_records = len(df)
```

Generated Test Report (data_quality_report.html):

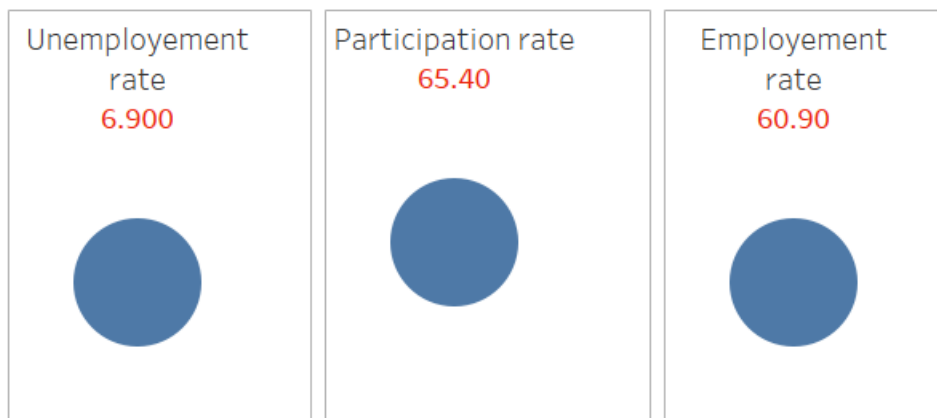


7. Data Analysis and Visualization

This section examines Canada's labor market dynamics through visualizations created with Tableau. Each graph addresses specific questions about employment trends, regional variations, occupational demands, and sectoral shifts. The analysis reveals critical patterns in workforce participation, growth sectors, and geographic imbalances, providing actionable insights for policymakers, educators, and businesses.

7.1 Data Visualizations

1. Canada's Labour Force Snapshot (KPI Cards)



Question Answered: What are the key indicators of current Canada's labour market performance?

Analysis: This KPI-style visualization highlights three headline metrics:

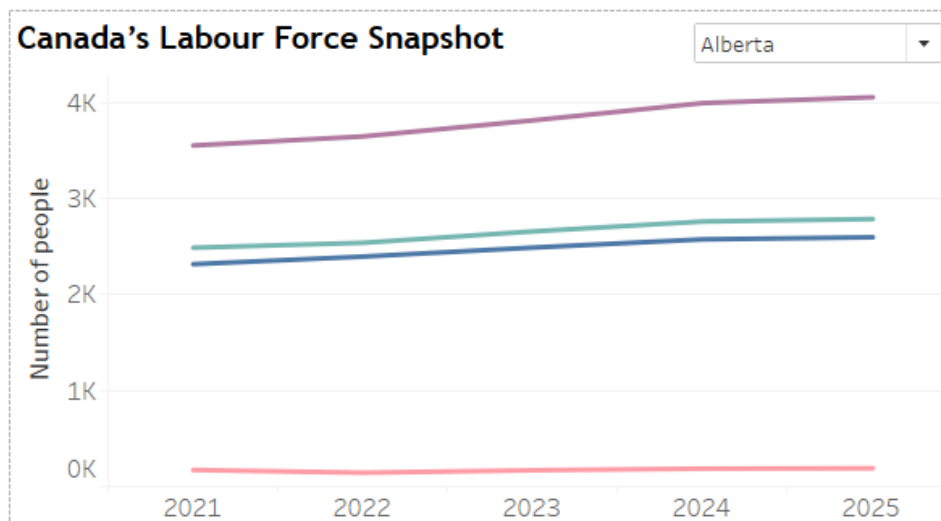
- Unemployment Rate: 6.9%
- Participation Rate: 65.4%
- Employment Rate: 60.9%

These figures provide a concise overview of the labour market. The unemployment rate suggests moderate slack, while the participation rate indicates how actively the population is engaged in the workforce.

Usefulness:

- Offers a quick reference for policymakers and economists to assess labour market health
- Supports regional and national planning for employment programs and resource allocation.

2. Labour Force Trends (2021–2025)



Question Answered: How has Canada's labour force evolved over the past five years?

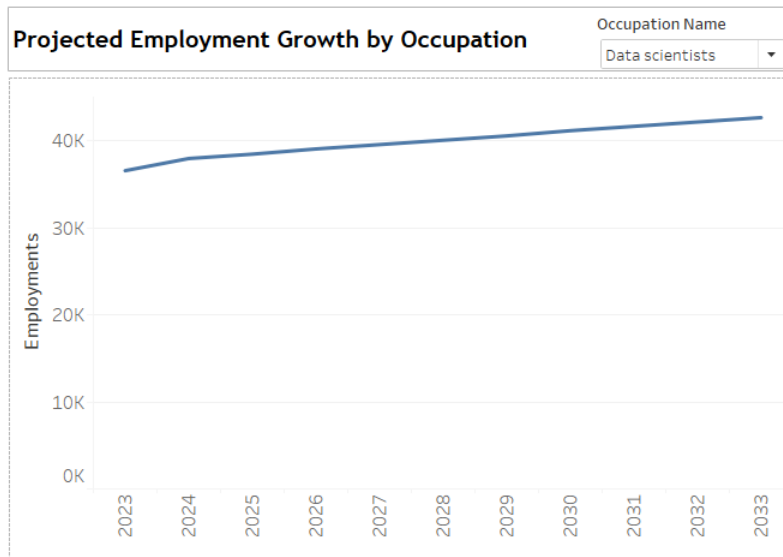
Analysis: This time-series graph compares employment, unemployment, and population metrics from 2021 to 2025. Employment steadily increases, while unemployment shows a slight decline, suggesting economic recovery and job creation. The participation rate remains relatively stable, indicating consistent workforce engagement.

Usefulness:

- Helps policymakers assess labour market health over time.

- Useful for identifying years with significant shifts in job availability.

3. Projected Employment Growth by Occupation (Ontario, 2021–2025)



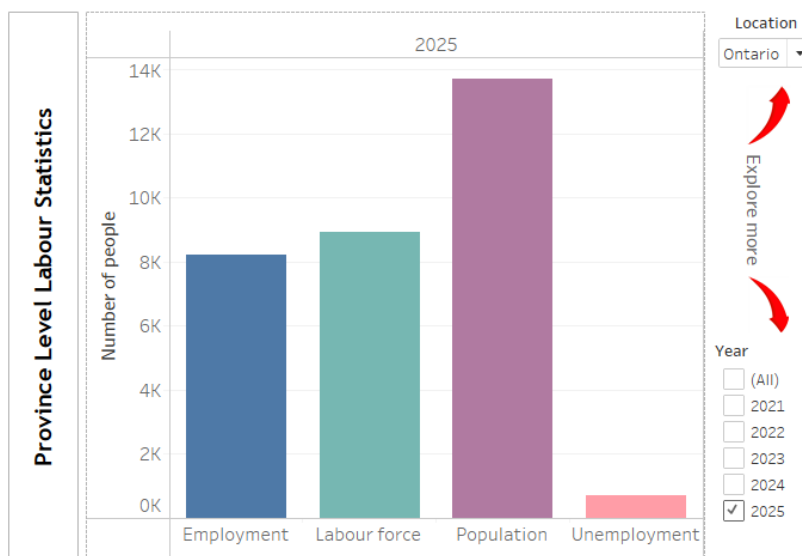
Question Answered: Which occupations are expected to grow in Ontario?

Analysis: The bar graph shows a clear upward trend in employment numbers across various occupations in Ontario.

Usefulness:

- Assists job seekers in targeting high-growth occupations.
- Enables educators and training institutions to align programs with market needs.
- Informs government strategies for workforce development.

4. Province-Level Labour Statistics (2025)



Question Answered: How do employment metrics vary across Canadian provinces?

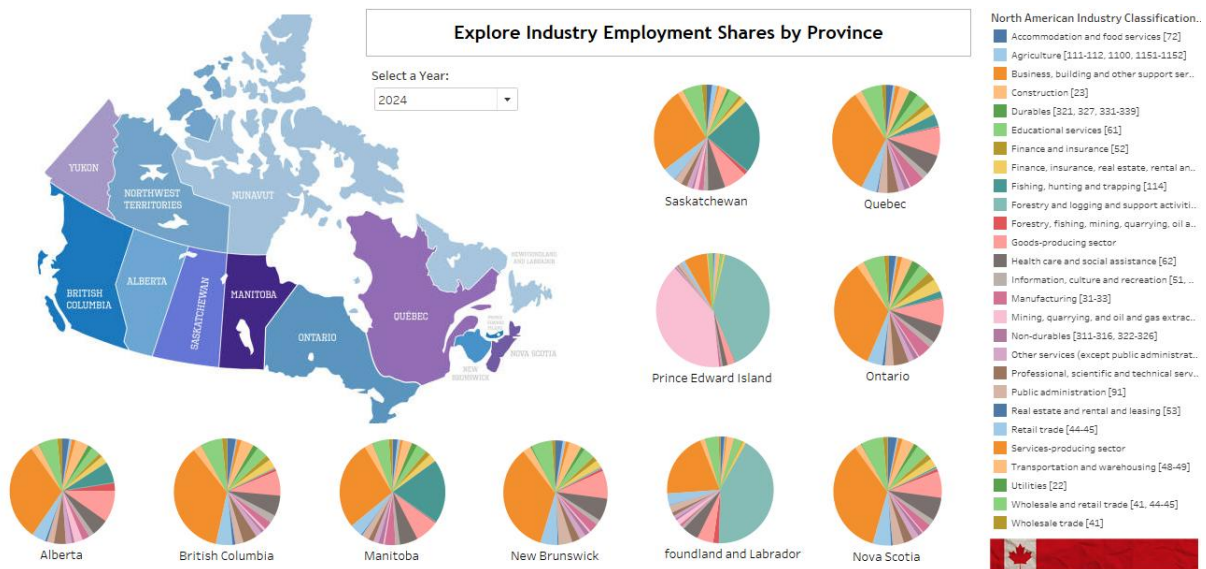
Analysis:

This map-based visualization displays employment rate, participation rate, and unemployment rate by province. Ontario and Alberta show high employment rates, while provinces like Prince Edward Island and Nova Scotia have higher unemployment.

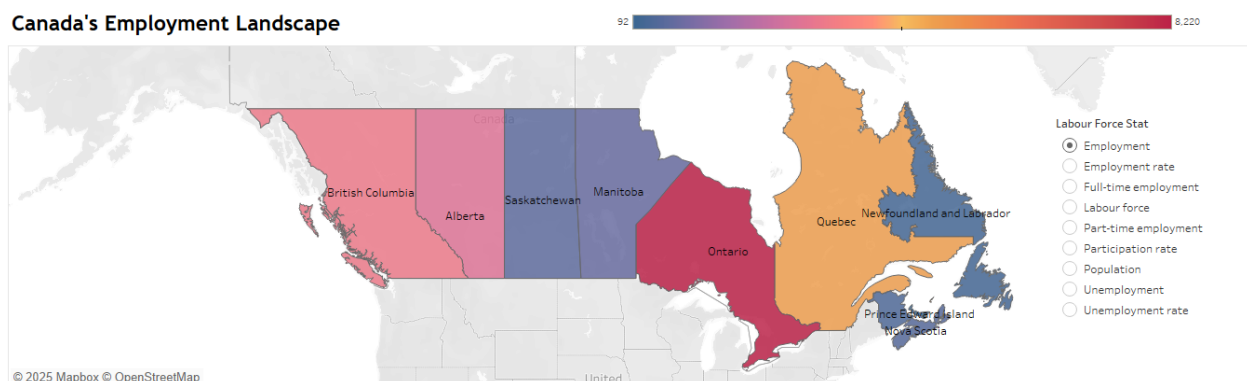
Usefulness:

- Enables regional comparison for targeted policy interventions.
- Helps businesses identify provinces with strong labour pools.
- Supports migration and relocation decisions for workers.

5. Industry Employment Shares by Province



Canada's Employment Landscape



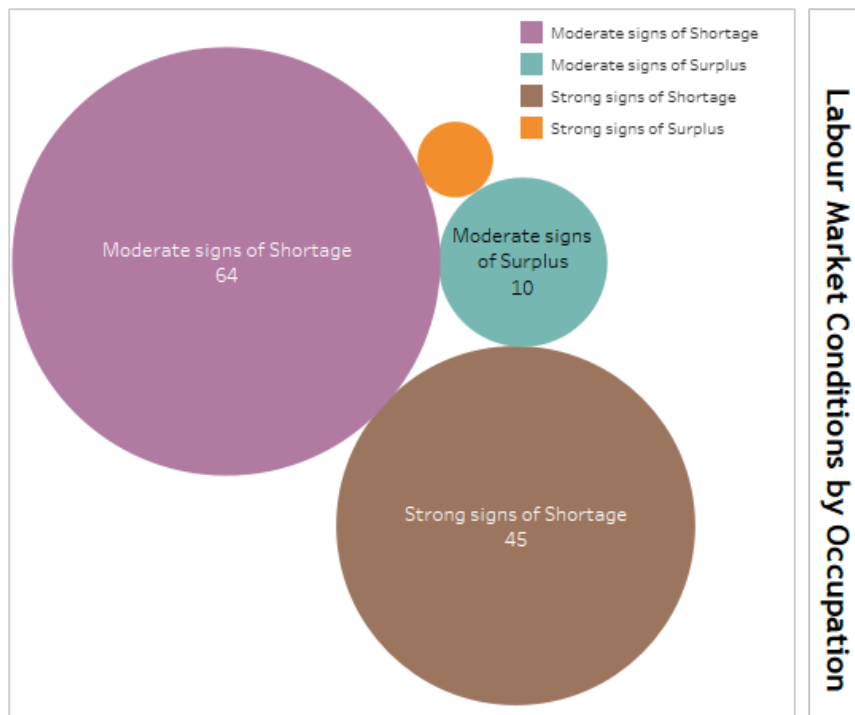
Question Answered: Which industries dominate employment in each province?

Analysis: This stacked bar chart breaks down employment by industry sector (e.g., healthcare, retail, manufacturing). Services-producing sectors dominate across most provinces, especially in Ontario and Quebec.

Usefulness:

- Guides investment decisions by highlighting dominant sectors.
- Helps job seekers understand provincial industry strengths.
- Informs provincial economic development strategies.

6. Labour Market Conditions by Occupation (Job Postings)



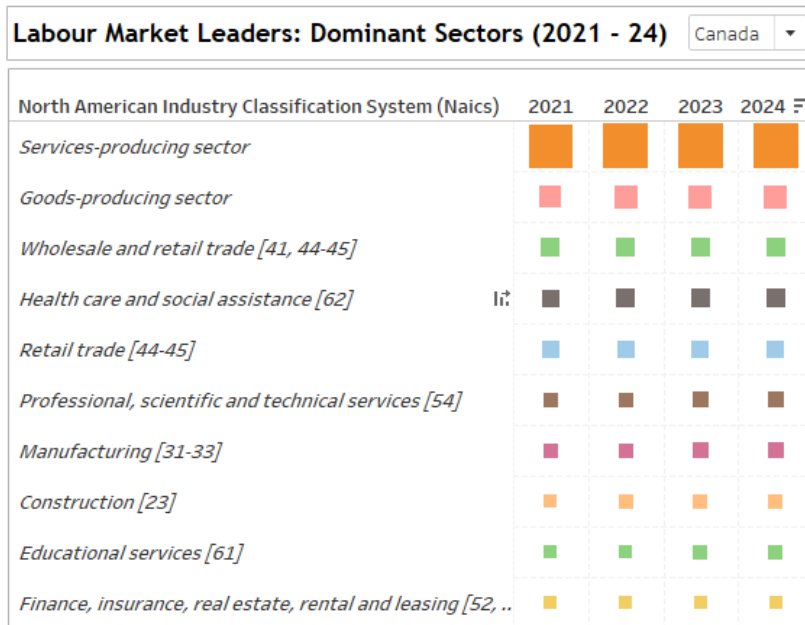
Question Answered: Which occupations show signs of shortage or surplus?

Analysis: This multi-bar graph categorizes occupations based on market conditions. Roles like restaurant managers and food service supervisors show strong signs of shortage, while others indicate surplus.

Usefulness:

- Help employers adjust hiring strategies.
- Informs training and reskilling programs.
- Supports labour market equilibrium by identifying gaps.

7. Labour Market Leaders - Dominant Sectors



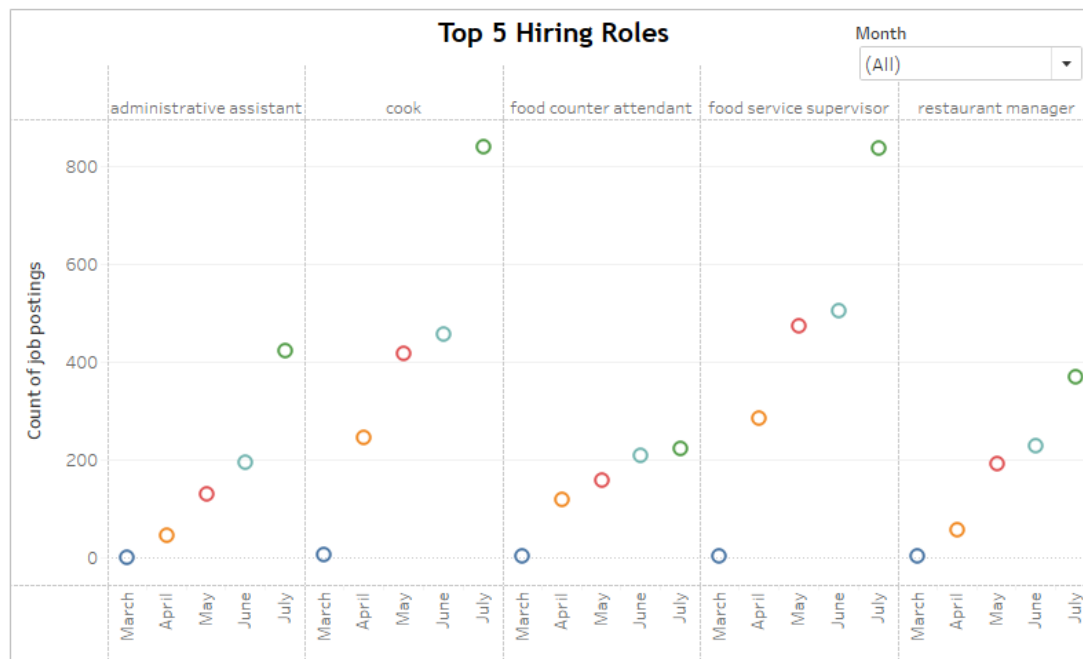
Question Answered: Which sectors are showing the strongest labour market activity over time?

Analysis: A stacked bar series depicts yearly signals of high employment sectors – top 10.

Usefulness:

- Helps forecast hiring needs and supply-demand imbalances.
- Encourages training programs in shortage areas.
- Aids economic diversification planning

8. Top 5 Hiring Roles



Question Answered: Which occupations saw the most job postings across Canada from March to July?

Analysis: This scatter plot tracks monthly job posting counts for five roles: Administrative Assistant, Cook, Food Counter Attendant, Food Service Supervisor, and Restaurant Manager. Each role saw a consistent increase in demand from March to July, with Administrative Assistant reaching nearly 1000 postings by July. Food-related roles also showed notable surges, especially in early summer months.

Usefulness:

- Helps predict seasonal hiring patterns across key service occupations.
- Informs career transitions into high-demand, entry-level roles.
- Supports employment programs aimed at boosting hospitality sector recovery.

9. Canada's Hottest Hiring Provinces



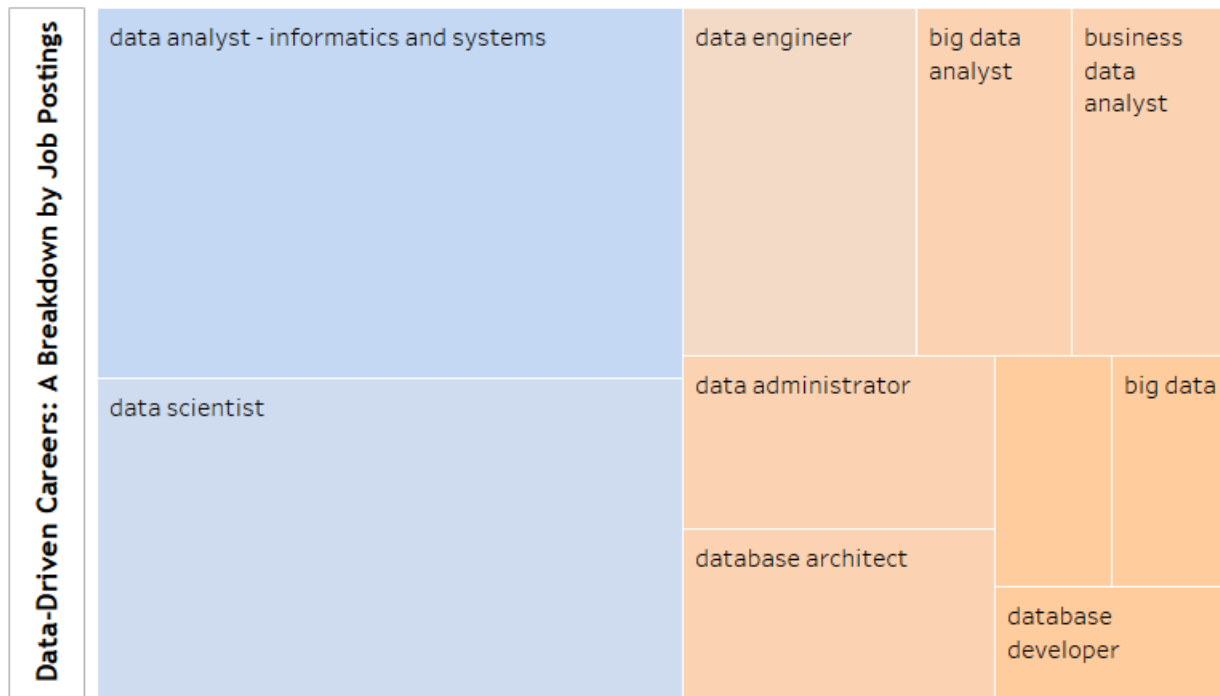
Question Answered: Which provinces are leading in job creation?

Analysis: This heatmap highlights provinces with the highest hiring activity. Ontario, Alberta, and British Columbia emerge as top hiring regions.

Usefulness:

- Assists job seekers in targeting high-opportunity regions.
- Informs interprovincial migration and housing policies.
- Supports regional economic planning.

10. Data-Driven Careers – Job Postings by Role



Question Answered: Which data-related roles are in high demand?

Analysis: This bar chart shows job postings for roles like data analyst, data engineer, and data scientist. Data analyst and business data analyst roles dominate, reflecting the growing importance of data skills.

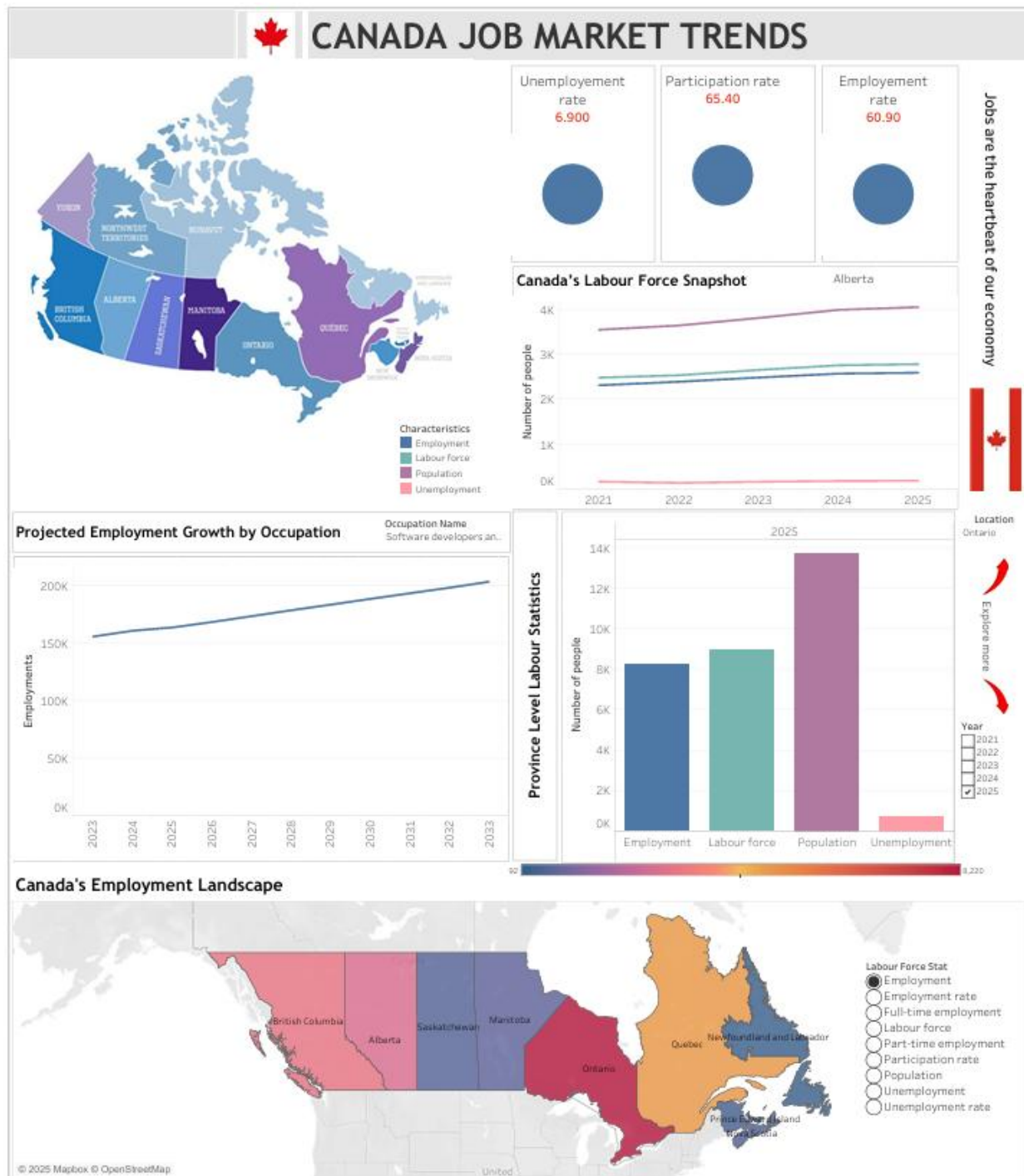
Usefulness:

- Guides students and professionals toward in-demand tech roles.
- Help recruiters focus on high-volume hiring areas.
- Supports curriculum development in data science programs.

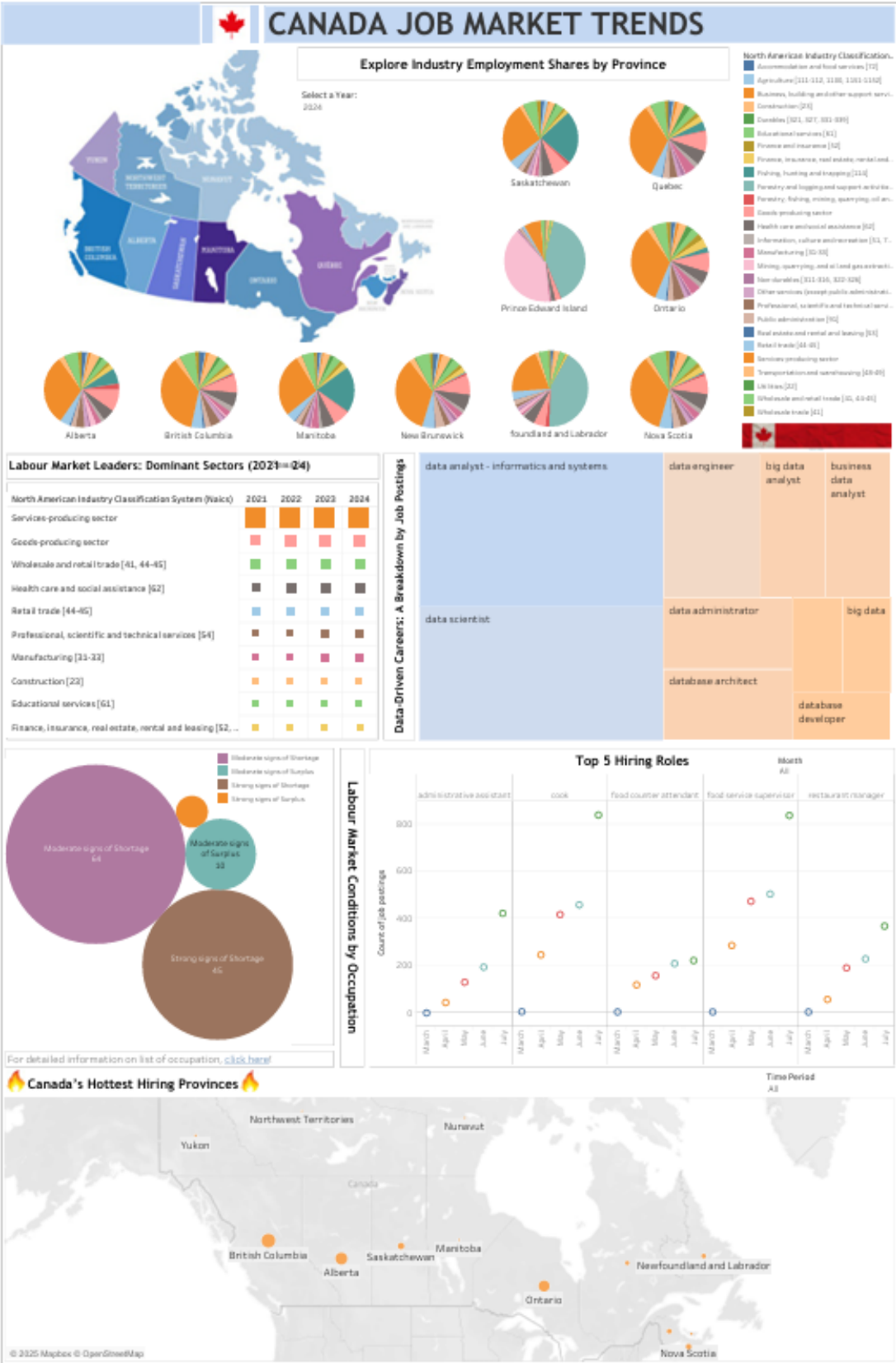
Dashboards

Two dashboards are implemented considering the variety of data available.

1. General Trends on labour statistics



2. Trends on jobs and industry



Filters can be applied for timeframe, occupations, industries and provinces. Despite Tableau's limitation in real-time data processing, these filtered visuals provide actionable intelligence for building a resilient, prioritizing high-impact interventions were shortages bite hardest.

8.DevOps

The project incorporates several DevOps principles to streamline the workflow from data collection to visualization. While it's not a full-scale production deployment, the project demonstrates a lightweight and effective DevOps approach focused on automation, integration, and reproducibility.

Version control is managed through our GitHub repository - [SENG8081-S25-Team1](#), which contains all source code, scripts, and visualization assets. Branching and pull requests support collaborative development, code reviews, and a continuous history of changes. This ensures transparency and team-wide accountability.

Automation plays a central role in our data pipeline. Python scripts are used to download datasets from Statistics Canada and Open Canada, scrape live job postings from Job Bank, and clean and transform the data using Pandas. These automated processes reduce manual effort and ensure consistent, reproducible results across multiple data sources.

For data integration and storage, we use MongoDB as our central database. It supports both historical and real-time datasets and is designed for scalability. The MongoDB BI Connector allows seamless integration with Tableau, enabling dynamic dashboards that reflect the latest labour market trends.

To automate our data pipeline and ensure timely updates, we implemented a background scheduling system using Python's `apscheduler` library. This scheduler orchestrates the execution of key modules at defined intervals using `CronTrigger`, allowing us to run data collection, job scraping, data loading, cleaning, and quality checks without manual intervention.

To further strengthen our DevOps capabilities, we plan to introduce enhancements such as automated CRON jobs or GitHub Actions for scheduled data updates, Dockerized ETL pipelines for consistent environment setup, and deployment of dashboards via Tableau Server or auto-refresh and broader accessibility.

In summary, our project applies DevOps principles to automate data workflows, maintain clean version control, and deliver timely insights making it scalable, maintainable, and ready for future expansion.

9. Extension

To scale this project in the future, we plan to enhance the system in the following directions:

Storage - MongoDB Atlas migration

As data volume increases, we plan to migrate from our current local Docker setup to MongoDB Atlas, a fully managed cloud service. This will give us access to features like automatic scaling, backups, and high availability. Atlas also includes built-in tools for monitoring, full-text search, and basic data visualization, which will help us manage a larger dataset efficiently and securely.

Backend system flexibility

We aim to build a more dynamic and configurable backend. In future versions, the admin dashboard will support adding or removing data sources without code changes. We'll also allow users to configure or adjust data collection schedules, cron jobs directly from the UI. This flexibility will make it easier to adapt to new data requirements or sources as the project grows.

Trend prediction and forecasting

Based on historical data, we plan to introduce simple predictive analytics features. These may include trend forecasting, such as identifying upcoming popular topics or patterns using time-series models or machine learning. This shift from descriptive to predictive analytics will help users gain forward-looking insights rather than just viewing current or past data.

Storage estimates and scaling strategy

With expected data growth from an existing storage, we'll leverage MongoDB's document-horizontal scaling to support this growth. To maintain performance, we will also consider using **indexing, cold storage for older data, and retention policies** that balance storage cost and historical value.

Real-Time Visualization

We plan to integrate Tableau with live MongoDB Atlas connections to deliver interactive, auto-refreshing dashboards. Additional visualization platforms like Power BI, Streamlit, Dash, and Grafana will provide complementary insights across data usage, system health, and backend performance.

10. Allocation Project Team Roles

Our team collaboratively assigned roles to ensure each member could focus on their strengths and contribute meaningfully to the project.

Role	Team Member	Responsibilities
Team Lead	Anuroopa B	Oversaw project direction, coordinated meetings, managed README, generated visualizations and ensured deadlines were met.
Documentation Manager	Bin Hu	Worked on Data Quality, testing and project documentation, ensured clarity and completeness.
DevOps/Git Manager	Chen Ce	Maintained GitHub repo, handled version control, managed branches and merges. Worked on scraper python code and batch processing.
Research & Data Engineer	Xiaoman Yang	Collected and cleaned datasets, implemented data pipelines and preprocessing.

11. Project Timeline

Date	Deliverable	Responsible
May 20	Data Researched and Collected	Team
June 4	Data cleaning	Anuroopa, Chen, Xiaoman
June 17	Data storage and maintenance	Bin, Xiaoman, Chen
Jun 18	Documentation	Bin, Anuroopa
July 24	Data Analysis and Visualization	Anuroopa, Chris
July 24	Data Quality Checking	Bin, Xiaoman
July 26	Final Adjustments made and checked	Anuroopa, Chris

July 29	Final Report and Presentation Slides	Team
July 31	Process and Report Due at 11:59pm	Team

References

- [1] Statistics Canada. (2025). *Employment by industry, monthly, unadjusted for seasonality* (Table 14-10-0287-03). Government of Canada.
<https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410028703>
- [2] Statistics Canada. (2025). *Labour force characteristics by age group and sex, monthly, unadjusted for seasonality* (Table 14-10-0023-01). Government of Canada.
<https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1410002301>
- [3] Employment and Social Development Canada. (2025). *Canadian Occupational Projection System (COPS) – Job openings by occupation (2023–2033)*. Open Government Portal.
<https://open.canada.ca/data/en/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/da1135c5-a1df-4a07-a81e-de308ae6cce6>
- [4] Employment and Social Development Canada. (2025). *Assessment of recent labour market conditions by occupation (2021–2023)*. Open Government Portal.
<https://open.canada.ca/data/en/dataset/e80851b8-de68-43bd-a85c-c72e1b3a3890/resource/ff795791-0728-4dcf-8fdd-d7ba9e1210b8>
- [5] Government of Canada. (2025). *Job Bank – Job search*.
<https://www.jobbank.gc.ca/jobsearch/>
- [6] IBM. (2022, September 23). *What is data quality?* IBM.
<https://www.ibm.com/think/topics/data-quality>
- [7] Natural Resources Canada. (2025, January 8). *Data Integration and Analysis*. Satellite imagery, elevation data, and air photos. Government of Canada.

<https://natural-resources.canada.ca/maps-tools-publications/satellite-elevation-air-photos/data-integration-analysis>

- [8] Tableau. (n.d.). *Build a basic view to explore your data*. In *Tableau Desktop and Web Authoring Help*. Retrieved July 31, 2025, from https://help.tableau.com/current/pro/desktop/en-us/getstarted_buildmanual_ex1basic.htm
- [9] The Data Signal. (2025, April 9). *Let's build a data quality checker in Python (stepbystep tutorial)* [Video]. YouTube. <https://www.youtube.com/watch?v=1kpZQpoxCWI>
- [10] MongoDB, Inc. (2025). *MongoDB BI Connector* (Version 2.14). <https://www.mongodb.com/docs/bi-connector/>